

L05: Fundamentals of Machine Learning

CSci 560 Deep Learning w/ Python (Chollet) Ch. 5

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

L05.1 Generalization: The goal of machine learning

CSci 560: Deep Learning w/ Python (Chollet) Ch. 5.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Generalization: The goal of machine learning

Fundamental Issue of ML

The fundamental issue in machine learning is the tension between optimization and generalization.

- After just a few epochs of training, performance on never-before-seen data starts to diverge
 - The models start to **overfit**
- **Optimization** refers to the process of adjusting a model to get the best performance possible on the training data.
- **Generalization** refers to how well the trained model performs on data it has never seen before.

Underfitting and Overfitting

- **underfit** the network hasn't yet modeled all relevant patterns in the training data.
- Eventually validation loss begins to increase while training loss continues to improve
 - Generalization has stopped improving, validation metrics stall and then degrade
- The model is **overfitting**
 - It is learning patterns specific to the training data (noise)

Overfitting is particularly likely to occur when your data is noisy, if it involves uncertainty, or if it includes rare features.

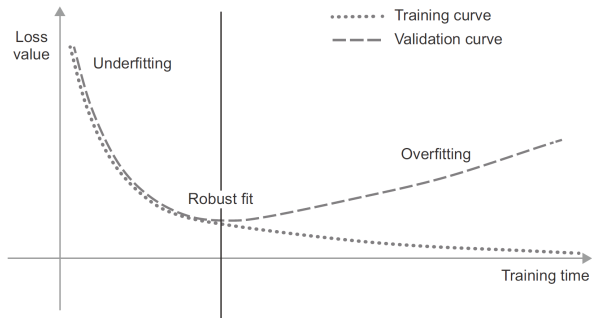


Figure 1: Canonical overfitting behavior. (Chollet, 2021, pg.122)

Noisy Training Data

- In real-world datasets, inputs are often **ambiguous** or **invalid**.
- Even worse, some of the data will be **mislabeled**.
- If a model goes out of its way to incorporate such outliers, its generalization performance will degrade.

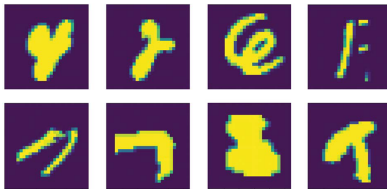


Figure 2: Weird MNIST training samples, what are these? (Chollet, 2021, pg.123)

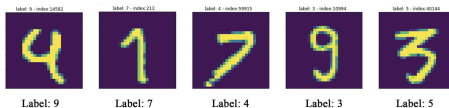


Figure 3: Mislabeled MNIST training samples? (Chollet, 2021, pg.123)

Dealing with Outliers: Robust fit vs. Overfitting

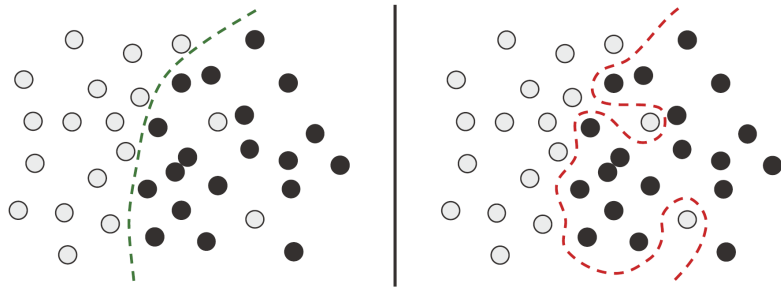


Figure 4: Dealing with outliers: robust fit vs. overfitting. (Chollet, 2021, pg.124)

- If a model goes out of its way to incorporate such outliers (ambiguous, mislabeled, noisy), its generalization performance will degrade.
- On the right, a model that overfits goes out of its way to incorporate examples that may be mislabeled.
- On the left, this model probably will generalize much better by ignoring outliers.



Ambiguous Features

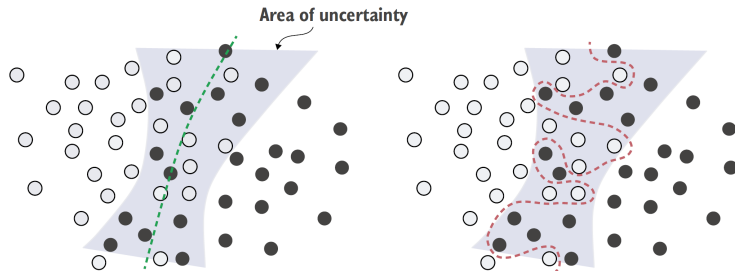


Figure 5: Robust fit vs. overfitting giving an ambiguous area of the feature space. (Chollet, [2021](#), pg.124)

- Not all data noise comes from inaccuracies
 - Even clean and correctly labeled data can be noisy.
- For example, the area of uncertainty might represent noise where it is difficult to differentiate between the two classes.
- A model that overfits on such noisy data will be too confident about ambiguous regions of the feature space.

Rare Features and Spurious Correlations

- Machine learning models trained on datasets that include rare features are highly susceptible to overfitting.
- Spurious correlations are present even in noise.
- Example training MNIST and adding noise vs. just 0's.

The Nature of Generalization in Deep Learning

- A deep learning model can be trained to fit anything, as long as it has enough representational power.
- You can get training loss on completely random noise. (see example)
- How can deep learning models generalize?
 - When learning noise like this example, are basically learning an ad hoc mapping between training inputs and targets, like a fancy dict

The Manifold Hypothesis

Manifold Hypothesis

The **Manifold**

Hypothesis posits that all natural data lies on a low-dimensional manifold within the high-dimensional space where it is encoded.

- Take MNIST dataset, total possible inputs is $256^{28 \times 28}$
- Very few of these inputs would look like a digit.
 - actual digits occupy a tiny *subspace* of the possible space
 - not a random subspace, it is **continuous**
 - all samples in the subspace are *connected* by smooth paths
- Handwritten digits form a **manifold** within the space of possible inputs.

The manifold hypothesis implies that:

- Machine learning models only have to fit relatively simple, low-dimensional highly structured subspaces within their potential input space.
- It's always possible to **interpolate** between two inputs.

The ability to interpolate between samples is the key to understanding generalization in deep learning.

Interpolation as a Source of Generalization

- You can make sense of the **totality** of the space using only a **sample** of the space.
- Interpolation along a manifold is different from linear interpolation in the parent space.
- Interpolation can only help you make sense of things that are close to what you've seen before.



Manifold interpolation
(intermediate point
on the latent manifold)



Linear interpolation
(average in the encoding space)

Figure 6: Difference between linear interpolation and interpolation on the latent manifold. Every point on the latent manifold of digits is a valid digit, but the average of two digits usually isn't. (Chollet, [2021](#), pg.124)



Why Deep Learning Works



Figure 7: Uncrumpling a complicated manifold of data. (Chollet, [2021](#), pg.130)

- A deep learning model is a tool for uncrumpling paper balls, that is for disentangling latent manifolds.
- A deep learning model is basically a very high-dimensional curve
 - smooth and continuous (since differentiable)
 - curve fitted to data via gradient descent (smoothly and incrementally)

Why Deep Learning Works

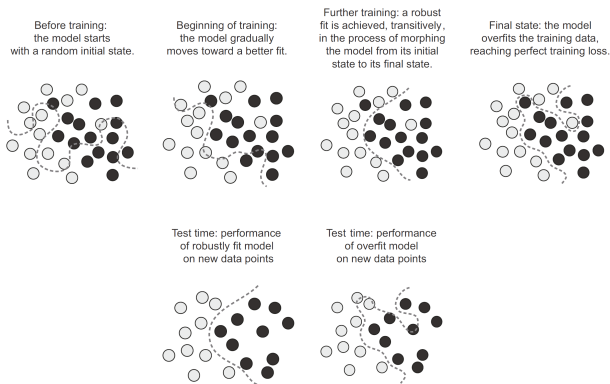
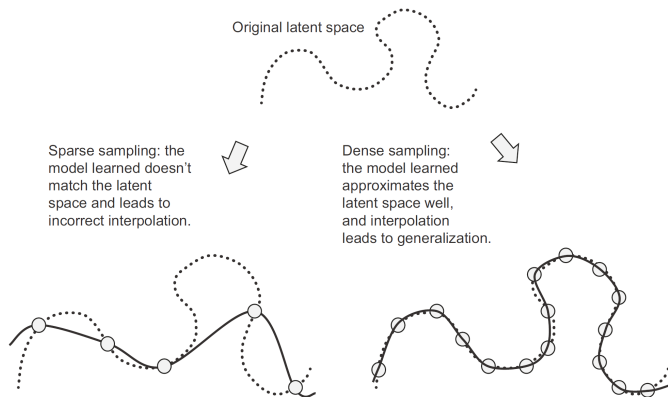


Figure 8: Going from a random model to an overfit model, and achieving a robust fit as an intermediate state. (Chollet, [2021](#), pg.131)

- The curve involves enough parameters that it could fit anything (even noise)
- **At some intermediate point during training the model roughly approximates the natural manifold of the data.**
- Deep learning models implement a smooth, continuous mapping from inputs to outputs. (differentiable)
- Deep learning models structured in a way that mirrors *shape* of the information in training data.



Training Data is Paramount



- The power to generalize is a consequence of the natural structure of your data.
- Data curation and feature engineering are essential.
- For a model to perform well **it needs to be trained on a dense sampling of its input space.**
- The best way to improve a deep learning model is to train it on more data or better data.

Figure 9: A dense sampling of the input space is necessary in order to learn a model capable of accurate generalization. (Chollet, [2021](#), pg.131)

L05.2 Evaluating Machine Learning Models

CSci 560: Deep Learning w/ Python (Chollet) Ch. 5.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

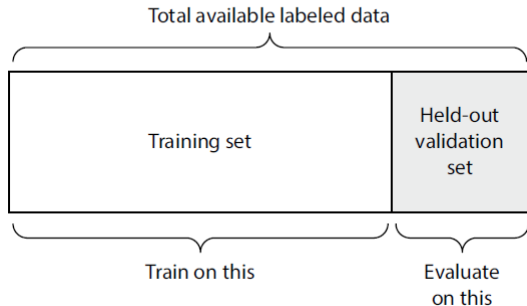
Training Validation and Test sets

Train / Validate / Test

You can only control what you can observe

- It's essential to be able to reliably measure the generalization power of your model.
- You of course need some **training data** to train the model on, this is supervised learning.
- Developing a model always involves tuning its configuration.
- You need a feedback signal of the model performance while training: **validation data**
- Can **overfit to the validation set**, thus need a final **test set**

Simple Holdout Validation



- Simple holdout validation is what we have mostly been using so far
- You can ask `keras fit method` to hold out some random part of the test data for validation for you.
- In order to prevent information leaks, you shouldn't tune your model based on the test set.

Figure 10: Simple holdout validation split. (Chollet, [2021](#), pg.134)

K-Fold Cross-Validation

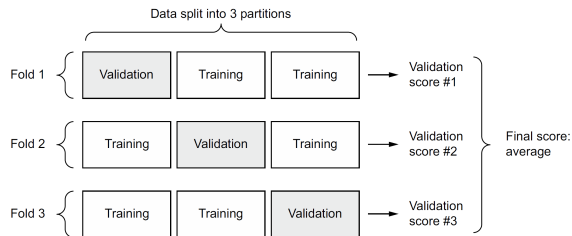


Figure 11: K-fold cross-validation with $K=3$ folds (Chollet, 2021, pg.135)

- If little data is available then your validation and test sets may contain too few samples to be statistically representative of the data.
- If different random shuffling rounds of data yield very different measures of performance, then you have this issue.
- K-fold validation and iterated K-fold validation can address this problem.
- Split data into K partitions of equal size.
- For each partition i train a model on the remaining $K-1$ partitions and evaluate on partition i

Iterated K-Fold Validation with Shuffling

- When have very little data available and need to evaluate model as precisely as possible.
- Apply K-fold cross-validation multiple times
 - Helpful if can call a K-fold with random shuffle as a reusable function.
- Final score is the average of the K-fold iterations (where are averages over K trainings).
- Expensive, end up training $P \times K$ models where P is number of K-fold iterations and K is number of folds used each time.

Beating a Common-Sense Baseline

- The only feedback you have is your validation metrics.
- Particularly important to tell whether or not you are actually able to predict anything at all or not.
- Before you start working with a dataset, you should always pick a trivial baseline that you'll try to beat.
 - If you beat that threshold, you'll know your model is at least doing something.
- If you have a binary classification problem where 90% of samples belong to class A, a classifier that always picks A achieves 90% accuracy.
- For classification, picking at random is often a basic baseline to beat.
- For regression, always guessing the mean or median output value is a low baseline.

Things to Keep in Mind about Model Evaluation

- **Data representativeness** You want both your training set and test set to be representative of the data at hand. For this reason you usually should at least **randomly shuffle** your data before splitting it into training and test sets.
- **The arrow of time** On the other hand, if you are doing time series prediction, you should **NOT** randomly shuffle your data before splitting it, this will cause **temporal leak**. Instead always make sure all data in test set is *posterior* to the data in the training set.
- **Redundancy in your data** Be careful of repeated data points appearing multiple times in a dataset (not uncommon). If you split a repeated sample and end up with it in the train and test set, you'll be leaking information and essentially testing with part of the data you trained with.

Having a reliable way to evaluate the performance of your model is how you'll be able to monitor the tension at the heart of machine learning—between optimization and generalization, underfitting and overfitting.

L05.3 Improving Model Fit

CSci 560: Deep Learning w/ Python (Chollet) Ch. 5.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

L05.4 Improving Generalization

CSci 560: Deep Learning w/ Python (Chollet) Ch. 5.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Generalization in a deep learning model originates from the latent structure of your data. (You are only as good as your data).

- Make sure you have enough data.
 - You need a **dense sampling** of the input-cross-output space.
- Minimize labeling errors
 - visualize your inputs and check for anomalies and outliers, proffread your labels.
- Clean your data and deal with missing values.
- Do feature selection, analyze correlation or importance of features in a decision tree.

- **Feature Engineering** is the process of using your own knowledge about the data to make the algorithms work easier
- The essence of feature engineering: making a problem easier by expressing it in a simple way.

Using Early Stopping

In deep learning we always have models that are vastly overparameterized: they have way more degrees of freedom than the minimum necessary to fit to the latent manifold.

- **You never fully fit a deep learning model.**
- Finding the exact point during training where generalization (validation loss) starts to decrease is one of the most effective things you can do to improve generalization.

Regularizing your Model

Regularization techniques are a set of best practices that actively impede the model's ability to fit perfectly to the training data, with the goal of making the model perform better during validation.

Adding Dropout

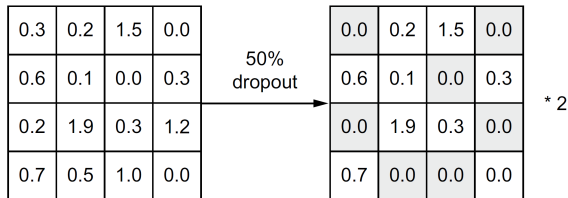


Figure 12: Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged. (Chollet, 2021, pg.150)

- **Dropout** is one of the most effective and most commonly used regularization techniques for neural networks.
- Dropout, applied to a layer, consists of randomly *dropping out* (setting to 0) a number of output features during training.
- The **dropout rate** is the fraction of the features that are zeroed out.
- At test time, we scale down output by the dropout rate.
- The core idea is that introducing noise in the output values of a layer can break up happenstance patterns that aren't significant.

Chollet, F. (2021). *Deep learning with python* (second). Manning.